

Árboles

Un **árbol** es una estructura de datos no lineal que representa una jerarquía de nodos conectados, donde cada nodo tiene un único padre (excepto la raíz) y puede tener dos nodos hijos.

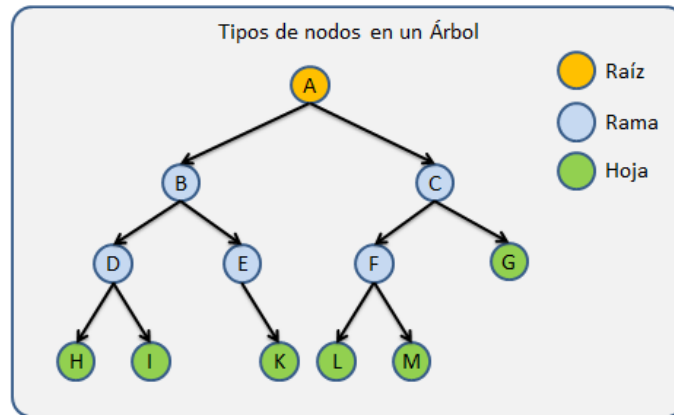
- **Nivel:** El nivel de un nodo es su distancia a la raíz. Por lo tanto:
 - Un árbol vacío tiene 0 niveles
 - El nivel de la raíz es 1
 - El nivel de cada nodo se calcula contando cuántos nodos existen sobre él, hasta llegar a la raíz + 1, y de forma inversa también se podría, contar cuántos nodos existen desde la raíz hasta el nodo buscado + 1.
- **Altura:** Se le llama altura al número máximo de niveles de un árbol.
- **Peso:** Es el número de nodos que tiene un árbol.
- **Orden:** El orden de un árbol es el número máximo de hijos que puede tener un Nodo. Es una constante que se define antes de crear el árbol.
- **Grado:** Número de hijos de un nodo y está limitado por el Orden, ya que este indica el número máximo de hijos que puede tener un nodo.
- **Camino:** Secuencia de nodos conectados dentro de un árbol.
- **Longitud del camino:** Cantidad de nodos que se deben recorrer para llegar desde la raíz a un nodo determinado.
- **Sub-Árbol:** Conocemos como Sub-Árbol a todo Árbol generado a partir de una sección determinada del Árbol
- **Hojas:** Nodos sin hijos

Operaciones

1. **Insertión:** agregar un nodo en la posición correcta
2. **Eliminación:** eliminar un nodo y reorganizar el árbol
3. **Recorridos:**
 - **Inorden (izquierda, raíz, derecha):** recorrido ordenado en árboles binarios de búsqueda.
 - **Preorden (raíz, izquierda, derecha):** útil para copiar estructuras.
 - **Postorden (izquierda, derecha, raíz):** usado en eliminación de árboles.

Aplicaciones

- **Bases de datos:** índices en bases de datos (Ejemplo: Árboles B-Trees en SQL)
- **Estructuras de archivos:** sistemas de archivos en computadoras
- **Búsqueda y ordenación eficiente:** árboles de búsqueda binaria



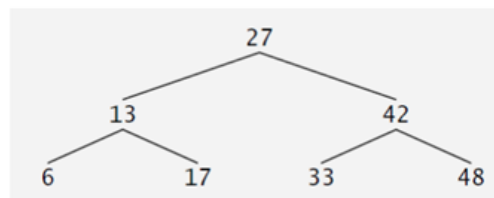
Un árbol con sus nodos distribuidos jerárquicamente

Tipos de árboles

- Árboles B
- Árboles B+
- Árboles B*

Preorden

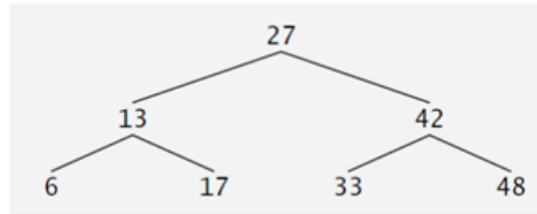
- Visitar la raíz (nodo actual)
- Recorrer el subárbol izquierdo en preorden
- Recorrer el subárbol derecho en preorden



Secuencia: 27 – 13 – 6 – 17 – 42 – 33 – 48

Inorden

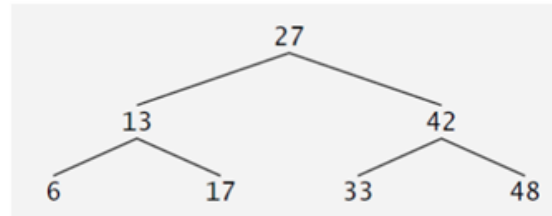
- Visita primero el izquierdo, luego el nodo, luego el derecho.
- Orden: Izquierdo → Raíz → Derecho
- Útil para árboles binarios de búsqueda, porque devuelve los elementos ordenados.



Secuencia: 6 – 13 – 17 – 27 – 33 – 42 – 48

Postorden (I-D-N)

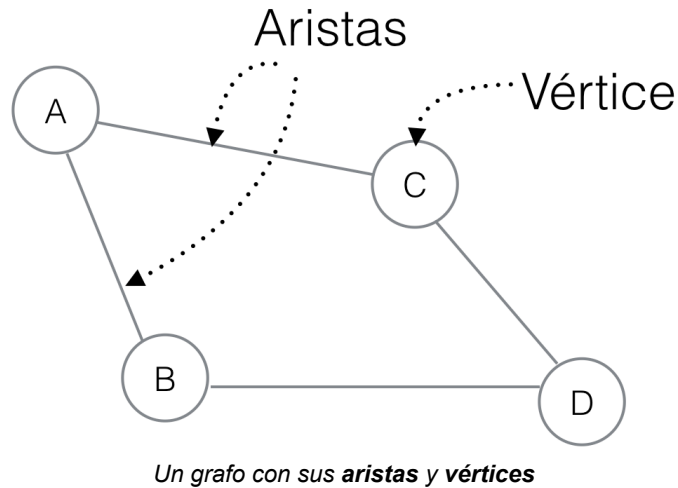
- Visita primero izquierdo, luego derecho, y al final el nodo.
- Orden: Izquierdo → Derecho → Raíz
- Útil para eliminar árboles o evaluar expresiones de forma postfija.



Secuencia: 6 – 17 – 13 – 33 – 48 – 42 – 27

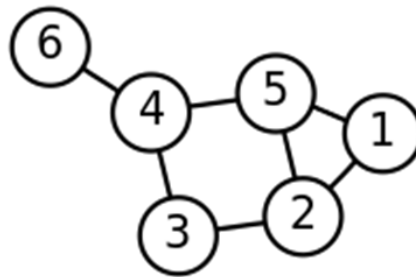
Grafos

Un **grafo** es una estructura de datos no lineal que se compone de **vértices** (nodos) y **aristas** (enlaces) que los conectan. Es ampliamente utilizado para modelar relaciones y conexiones en distintas aplicaciones. (es de tipo de dato abstracto TAD)



Formalmente, un grafo se define como $G = (V, A)$

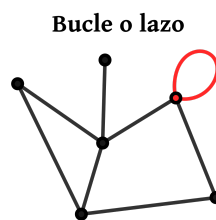
Siendo V un conjunto cuyos elementos son los vértices del grafo y A un conjunto cuyos elementos son las aristas, las cuales son pares (ordenados si el grafo es dirigido) de elementos en V .



Se llama **orden del grafo** G a su número de vértices, $|V|$

El **grado de un vértice** es el número de aristas que inciden sobre él. La suma de los grados de los vértices es igual al doble del número de aristas.

Un **bucle** es una arista que se relaciona al mismo nodo; es decir, una arista donde el nodo inicial y el nodo final coinciden.



Dos o más aristas son **paralelas** si relacionan el mismo par de vértices



Tipos de Grafos

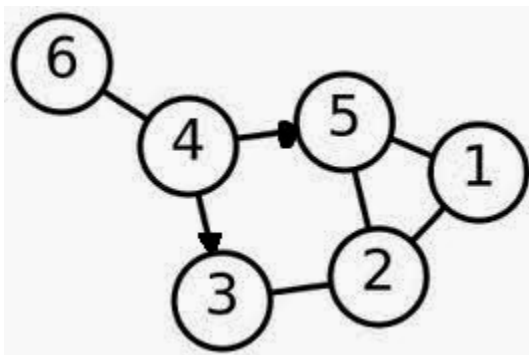
- ❖ **Grafo dirigido:** las aristas tienen dirección ($A \rightarrow B$, significa que se puede ir de A a B, pero no necesariamente al revés)



- ❖ **Grafo no dirigido:** la relación entre nodos es bidireccional ($A \leftrightarrow B$, significa que se puede ir en ambos sentidos)



- ❖ **Grafo mixto:** Un grafo mixto es aquel que se define con la capacidad de poder contener aristas dirigidas y no dirigidas. Tanto los grafos dirigidos como los no dirigidos son casos particulares de este



Propiedades

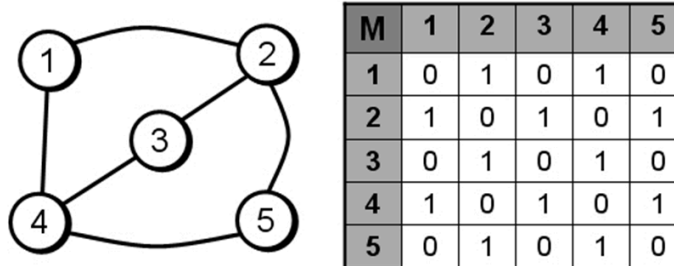
- ❖ **Adyacencia:** dos aristas son adyacentes si tienen un vértice en común, y dos vértices son adyacentes si una arista los une.



- ❖ **Incidencia:** una arista es incidente a un vértice si ésta lo une a otro.
- ❖ **Ponderación:** corresponde a una función que a cada arista le asocia un valor (costo, peso, longitud, etc.), para aumentar la expresividad del modelo. Esto se usa mucho para problemas de optimización, como el del vendedor viajero o del camino más corto.
- ❖ **Etiquetado:** distinción que se hace a los vértices y/o aristas mediante una marca que los hace unívocamente distinguibles del resto.

Representación

Matriz de adyacencia (MA): Se utiliza una matriz de tamaño $n \times n$ donde las filas y las columnas hacen referencia a los vértices para almacenar en cada casilla la longitud entre cada par de vértices del grafo. La celda $MA[i, j]$ almacena la longitud entre el vértice i y el vértice j . Si su valor es infinito significa que no existe arista entre esos vértices, y $MA[i, i] = 0$



Aplicaciones

- ❖ **Búsqueda de rutas más cortas:** algoritmo de Dijkstra, Google Maps
- ❖ **Detección de ciclos:** detección de dependencias en compiladores
- ❖ **Modelado de redes sociales:** representación de amigos y conexiones en Facebook
- ❖ **Lista de Adyacencia:** Se usa cuando el grafo es esparso (pocas conexiones)
- ❖ **Matriz de Adyacencia:** Se usa cuando el grafo es denso (muchas conexiones)